# ClimaTwin: Hardware & Test Guide (v2.1)

Team TRIANGLE · CIT Make-A-Thon Round 2 · Architect Directive

**Updated: 10 Nov 2026 (21:18 IST)**
Upload directly to ChatGPT / Codex for guidance

**INSTRUCTIONS FOR CHATGPT / AI ASSISTANT (READ FIRST WHEN UPLOADED)**

- **Actual Hardware:** The team does **NOT** have an SHT31. They have a working **ESP32 WROOM NodeMCU 38-Pin**, **1× DHT11 Sensor Module**, **2× DS18B20 Probes (now working)**, **4.7kΩ pull-up resistors**, and a **GL12 840-pt breadboard**.
- **Strict Integrity Rule:** NEVER write code that generates fake/random sensor readings in live mode.
- **Data Contract:** Firmware must print exactly 5 comma-separated values at 115200 baud: `ms,probe0_C,probe1_C,room_C,room_RH`.
- **Core Fit Rule:** Python's `twin/fit_tau.py` fits Newton cooling ONLY on `room_C` and rejects runs if a 3-min window is flatter than `-0.02 °C/min`.

**STATUS UPDATE — DS18B20 ONLINE:** Great work getting DS18B20 running! Keep your working pins/libraries and never generate random values in live mode. Before starting the 60-min AC runs, complete the **2-minute verification check** below so tonight's CSVs pass validation cleanly.

## 1. Real Hardware BOM & Pinout (₹1,281 Invoice INV2223-1599)

| Component | Qty | ESP32 Pinout | Verification Note |
|---|---|---|---|
| **ESP32 WROOM NodeMCU 38-Pin** | 1 | USB Type-C (115200 baud) | Use GPIO numbers in code (`27`, `26`), not `D4`. Avoid pins 34–39, 0, 2, 12, 15. |
| **DS18B20 Temperature Probes** | 2 | BOTH Yellow → `P27` (`GPIO27`, shared Col 24)<br>VCC (Red) → `3V3`, GND → `GND` | Ensure **both probes** share `P27` cleanly (count = 2) and never read `-127.00` or `85.00`. |
| **4.7 kΩ Pull-Up Resistor** | 1+ | Between `P27` (Col 24) and `3V3` | Single `4.7 kΩ` pull-up serves both shared DS18B20 probes. |
| **DHT11 Sensor Module** *(Replaces SHT31)* | 1 | Signal (`out` / `S`) → `P26` (`GPIO26`)<br>VCC → `3V3`, GND → `GND` | Match tiny `+ / out / -` labels on your board; no extra resistor needed. |
| **GL12 840-Point Breadboard** | 1 | — | **Note:** Power rails (+ / -) split at column 30; keep wiring left of Col 30. |

## 2. Pre-Test Verification Checklist (2 Minutes Before AC Run — Tasks 34 & 35)

1. **Confirm Count = 2:** Verify Serial Monitor prints `# DS18B20 count: 2` (both Supply & Return probes active, no `-127.00` or stuck `85.00` readings).
2. **Pinch Test & Cable Labels (Task 35):** Warm one probe between your fingers for 5 seconds—watch whether `probe0_C` or `probe1_C` rises, label cables **SUPPLY** / **RETURN**, and verify `PROBE_ROLES` in `config.py` matches!
3. **5-Min Ambient Offset + 5 Serial Lines (Task 34):** Leave both probes + DHT11 side-by-side at room temp for ~5 min, record average offset vs ambient, and paste **5 Serial lines** to Nithish (checks whether DHT11 prints `0.1°C` decimals like `28.40` or whole integers like `28.00`).
4. **Mechanical Strain Relief:** Loop jumper wires securely (electrical tape / zip-tie near the breadboard) so taping probes onto the AC vent does not tug loose a breadboard pin mid-run.

## 3. Reference Firmware Contract (ESP32 + 2× DS18B20 + DHT11)

**Already printing cleanly? Keep your working library/pins!** Only confirm Serial prints **5 comma-separated numbers** matching this exact contract at **115200 baud**:

```
#include <OneWire.h>              // Works cleanly on ESP32 Core v2.0.17 (or use OneWireNg on Core 3.x)
#include <DallasTemperature.h>
#include <DHT.h>

#define ONE_WIRE_BUS 27           // BOTH DS18B20 Yellow wires + single 4.7k pull-up to 3V3 (Col 24)
#define DHTPIN 26                 // DHT11 Signal (out/S) -> P26 (onboard pull-up included)
#define DHTTYPE DHT11

OneWire ow(ONE_WIRE_BUS);
DallasTemperature ds(&ow);
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  delay(1000);
  ds.begin();
  dht.begin();
  Serial.print("# DS18B20 count: ");
  Serial.println(ds.getDeviceCount());
  Serial.println("ms,probe0_C,probe1_C,room_C,room_RH");
}

void loop() {
  ds.requestTemperatures();
  float p0 = ds.getTempCByIndex(0);
  float p1 = ds.getTempCByIndex(1);
  float roomC = dht.readTemperature();
  float roomRH = dht.readHumidity();
```

```
  Serial.printf("%lu,%.2f,%.2f,%.2f,%.1f\n", millis(), p0, p1, roomC, roomRH);
  delay(5000);
}
```

**Step A: Verify & Label Probes (Task 35)**

1. Confirm `# DS18B20 count: 2` prints at startup.
2. Pinch one probe in your hand—whichever column rises is `probe0` or `probe1`. Label cables **SUPPLY** and **RETURN**.
3. Send Nithish **5 Serial Monitor lines** so we see if DHT11 outputs decimals ( `28.40` ) or integers ( `28.00` ), plus the 5-min side-by-side offset check!

**Step B: Live Capture Watch-Outs (No Fake Data)**

- Never replace missing/stuck values ( `-127` , `85` , `nan` ) with random numbers.
- If a probe drops mid-run, twist leads tightly onto solid male pins on `Col 24` ( `P27` ) and keep the `4.7 kΩ` pull-up firm to `3V3` (a second `4.7 kΩ` in parallel = `2.35 kΩ` is optional if using long cables).
- If only 1 probe or coarse DHT11 ticks appear, send Nithish a wiring photo + 5 Serial lines immediately.

## 4. Tonight's Fast, Reliable 3-Phase AC Test Protocol

1. **Pick a Small, Enclosed Room:** Close all doors, windows, and curtains. Keep AC setpoint identical (e.g. **18°C**) and fan speed fixed across all 3 runs. Note AC nameplate kW (photo) for the PDF report.
2. **Mount Safely:** Tape **Supply Probe** 5–8 cm in discharge airflow (not touching plastic louvers). Tape **Return Probe** at top intake grille. Place **DHT11** mid-room (~1.5m high, away from direct cold draft and ESP32 heat).
3. **CRITICAL NIGHT-TIME RULE (Avoid Rejected Curves!):** Since it is night, the room cools easily. **Before starting EACH phase**, turn the AC OFF, open the door, and run the ceiling fan for ~20 minutes so the room warms back up to **28–30°C**. Our fitter rejects runs that drop less than `0.5°C` or plateau early!
4. **Run & Log All 3 Phases (20–25 min each):** Close Serial Monitor first, then run from repo root:

```
python -m logger.log_serial --port COMx --phase clean1 --setpoint-c 18 --duration-min 22
# Warm room back up (~20 min, door open + fan ON), power AC OFF, half-cover filter with cardboard & photo!
python -m logger.log_serial --port COMx --phase blocked --setpoint-c 18 --duration-min 22
# Remove cardboard, warm room back up (~20 min)
python -m logger.log_serial --port COMx --phase clean2 --setpoint-c 18 --duration-min 22
```

5. **Record 45s Demo Video Clip:** Film mounting the probes, clean run, sliding cardboard over the filter, and removing it.
6. **Validate & Push to GitHub:** Run `python tools/check_csv.py data/raw` and `python -m twin.fit_tau data/raw` , then push CSVs + offsets to GitHub.

## 5. Spec v2.1 Quick Reminder (Product Narrative)

**Updated Positioning:** Pro tools (Testo/measureQuick) verify repairs for ₹50k+ contractors; smart remotes (Sensibo/Cielo) just run hour timers. ClimaTwin is a **₹1,281 zero-mains Digital Twin** guiding households through **"Did cleaning the filter actually help?"** (Guided Experiment), exposing **"Cold Vent vs. Slow Room"**, and exporting a weather-adjusted Proof-of-Service / Pre-Visit Technician Brief.